The Cost of Security: Quantifying the Security-Performance Tradeoff in Linux DPDK Environments

Nikhil Jayakumar

December 15, 2025

Abstract

Modern high-performance networking environments use the Linux DPDK as a way for userspace applications to bypass the kernel for direct access to memory and network hardware. However, bypassing the isolation and security provided by the kernel presents a security risk that many applications do not account for. Our work formally quantifies this security gap from multiple perspectives, measuring performance and security of malicious packets, increased OSI validation checks, and a virtualized DPDK environment. Our work shows 2-4x slowdown for DPDK applications by integrating the same security that the Linux kernel provides. Furthermore, we allow DPDK developers to integrate granular level security checks into their project, allowing freedom to directly control the security-performance tradeoff they are willing to tolerate.
[1]

1 Introduction

The demand for high-throughput and low-latency networking is everlasting, especially with the rise of cloud computing and edge network services. Specialized hardware allows for high-bandwidth data transfers as well as Remote Direct Memory Access (RDMA) to accelerate heterogeneous workloads and network-driven applications. However, for these speeds to be realized, corresponding software needs to be processed at the same speed. To meet this requirement, traditional operating system networking stacks, which were designed around security and control, are often bypassed. An example of such technologies is the Linux Data Plane Development Kit (DPDK) which bypasses the kernel to eliminate the cost of user-kernel context switching, memory copies, and general purpose packet processing. This bypassing has allowed anywhere from 3-10x speedup [VT23] .

This endless demand for performance has often come at the cost of security. Kernel-level processing allowed for validations such as input/output virtual address translation, TCP/UDP checksum verification, and anti-spoofing checks. When bypassing the kernel, the user application is left vulnerable to malicious or erroneous data. Prior research has extensively quantified the performance gain of various kernel bypass technologies, but often fail to account for the lack of security sacrificed with such performance gains. By re-implementing these kernel security measures back into DPDK, this work shows whether the benefit of DPDK is lost when applications need to be secure and reliable. Our work allows developers to not choose between security and performance, and allows applications to integrate proper fine-grained security into their high-performance networking applications.

[1] Code and data available at: https://github.com/nikhiljayakumar/dpdk-security-cost

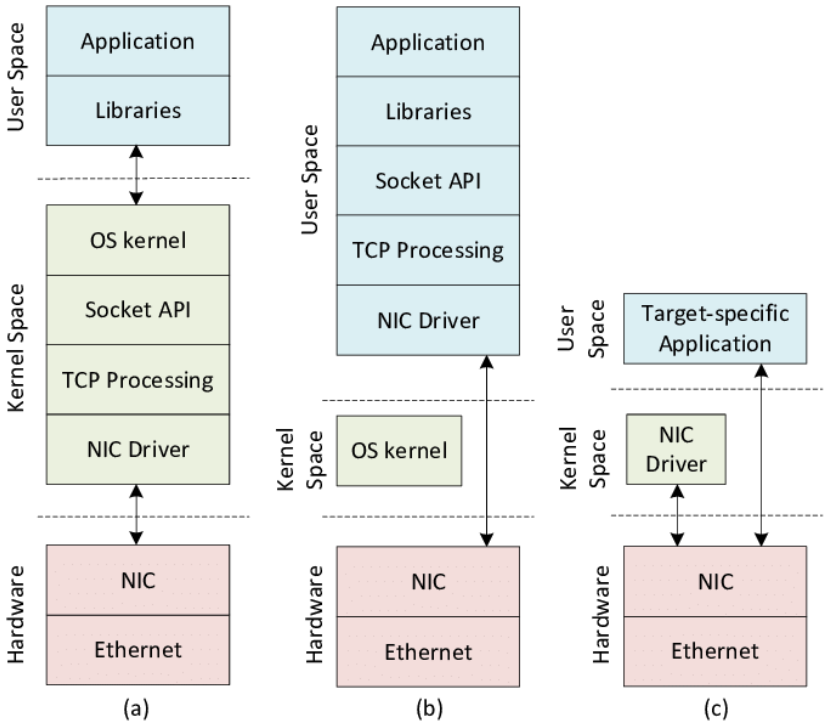


Figure 1: Example Kernel Bypass Architecture [GKCS22]

In order to correctly quantify the performance penalty seen by security protocols, we re-implement these same security protocols and measure the performance from various angles such as throughput, CPU cycles per packet, and latency. Furthermore, we quantify these penalties with granular levels of verification (e.g. verify just the TCP checksum). This granular approach provides a novel understanding of the security-performance tradeoff in Linux DPDK. Aside from specific protocols, we look into various methods of these security implementations, such as VirtIO offloading or completely virtual based checks. This systematic and granular measurement of security protocols presents a novel perspective on the true benefit of Linux DPDK for secure and performant applications.

To formally quantify this trade-off, we split our experiments into three research questions. First, we measure the throughput (packets per second) and per-packet latency of various levels of secure packet processing. Specifically, we measure 4 different levels including layer 3 verification and layer 4 Verification (discussed in section 3). Second, we measure the difference of Guest vs. Host Security protocols. We utilize the same levels of secure packet processing, but apply to both the host bare metal and the guest virtual machine. Finally, we measure the resilience of DPDK against malicious and erroneous packets. Not only can this corrupt and destroy user applications, but it could destroy DPDK performance gains due to cache misses or incorrect memory address translations.

The findings from this evaluation provide developers with tremendous insights into the benefits of DPDK and kernel bypass techniques for secure and reliable applications. For instance, developers can understand the specific performance-security tradeoff they are willing to tolerate for their application as well as design their application with specific guidelines about guest versus host based security validation. Future works include a more efficient secure kernel bypass or more security protocols added onto Linux DPDK. The remainder of this report details the motivation, the experimental setup and design, results and evaluation, related work, and conclusion.

2 Background and Motivation

The motivation of this project is twofold: quantifying the security-performance tradeoff as well as providing simple security checks for DPDK applications. This section details the background and motivation of kernel-bypass technologies as well as their need for enhanced security.

The Network Interface Card (NIC) is the first place incoming frames hit on a machine. The NIC

matches the layer 2 MAC address and verifies the checksum before making a direct memory access to move the frame into kernel memory. The NIC also has its own small section of memory where it stores circular queues for transmitting and receiving frames, known as TX and RX ring buffers respectively. These buffers simply store DMA descriptors and addresses of packets in memory. Next, the NIC triggers a kernel interrupt [Riz12].

This kernel interrupt is split into two halves. The top half is simply the interrupt service routine, designed to be quick and minimal. Therefore, the main goal of this application is to simply acknowledge the interrupt and schedule the bottom half to the CPU. The bottom half is what actually processes and verifies the packet. This includes multiple checksum verifications, layer 4 (TCP/UDP) processing, layer 3 (IP) processing, and finally sending the packet to the application's socket [GEW+15].

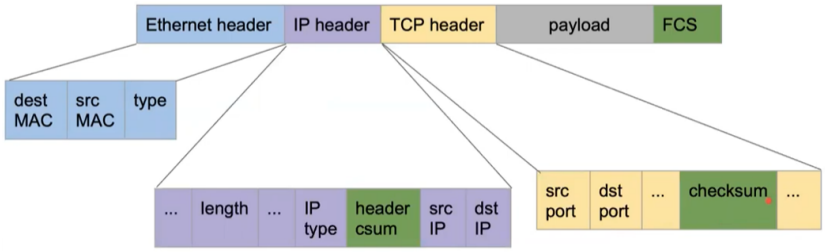


Figure 2: Network Packet Contents [SN16]

A complex and expensive operation done by the kernel is the checksum validation. For instance, the TCP checksum requires a re-construction of a "pseudo-header" which uses various layers of the packet header (IP addresses, message length). Along with the algorithmic complexity, this process can result in expensive memory access patterns. Although some NICs can offload this computation, it is often more secure to have it calculate it within the kernel. Other solutions involve offloading the checksum validation offloading on custom hardware (i.e. FPGA) [SRLBA18].

The kernel network stack is expensive due to its interrupt-driven nature and packet processing cost. With higher packet rates, the top half can be too expensive on the CPU such that there is little room left for the bottom half (known as "Receive Livelock") [Riz12]. Polling-mode device drivers, used by kernel-bypass technologies including DPDK, are much faster due to less kernel-user context switching overhead [ASVT18]. Kernel bypass technologies further improve by bypassing most of the "bottom-half" of the packet processing. It then forces the end application to handle any packet processing. Other kernel-bypass technologies such as Vector Packet Processing (VPP) extend DPDK by using batches (vectors) to amortize the cost of fetching network instructions for the CPU [BLM+18].

The obvious gap seen by these kernel-bypass technologies is the lack of security or validation done by packets. This forces the user application to perform these security checks, which can have adverse effects on not only the application but even the underlying OS/machine. Other works acknowledge the lack of isolation and security seen by kernel-bypass technologies [SAPK23], but many assume the end-application will handle these steps [Z+18, HZLM16, SLW+20].

This work bridges the gap between security and performance with a thorough analysis of integrating security in kernel-bypass technologies. Furthermore, this work will provide simple security checks for DPDK developers to integrate into their applications.

3 Proposed Design or Architecture

All the experiments were conducted on a single physical host that emulates a cloud Network Function Virtualization (NFV) environment. This allows for fine-grained control over the hardware and software along with accurate measurements of the CPU and virtual network devices.

The experiments for the first research question were run on a bare metal device with virtual network interface cards. Figure 1 shows a component diagram of the experimental setup showing both the physical hardware and the software running on it. Specifically, Ubuntu 24.04 was installed on a Ryzen 5 5600u CPU. DPDK was built from source and a modified version of the layer 2 Packet Forwarding (L2FWD) example was used. This example creates two ports and sends packets back and

forth between ports. The example was modified to generate packets if none were received, allowing this test to be a standalone baseline to measure packet processing speed. All further experiments were based off of this modified L2FWD test.

Our evaluation metrics mainly consist of performance, hardware usage, and security. Therefore, all experiments will be evaluated with 3 main metrics: latency, throughput, and CPU utilization. We will also be considering a qualitative, holistic threat model to evaluate security implications and trade-offs. Some experiments will go deeper into architectural metrics as the main metrics cannot give a proper understanding of the results.

The first research question adds a granular verification to the L2FWD test, and allows the user to pick what level of granularity they prefer. Figure 1 describes the different levels of verification we have tested. These levels will be measured and evaluated through the metrics described above. It can be assumed that increasing the level of verification will simply degrade the performance of DPDK. However, we hypothesize that DPDK will still be beneficial to the end user despite the security overhead.

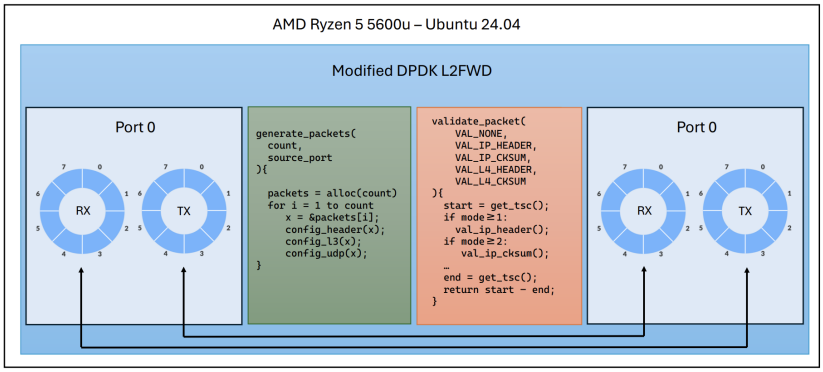


Figure 3: Modified DPDK L2FWD for Granular Packet Validation.

The second research question applies the same experimental setup but added a virtualization layer to measure the difference between Guest side validation and VirtIO offloads. This is done through a QEMU VM running on the machine, and L2FWD is ran on the VM. The host machine does not run any DPDK applications except generating packets to send to the VM and performs validation offloaded by the VM through VirtIO. We hypothesize that this simple change can drastically change our metrics due to general context switching and Host-VM communication overhead.

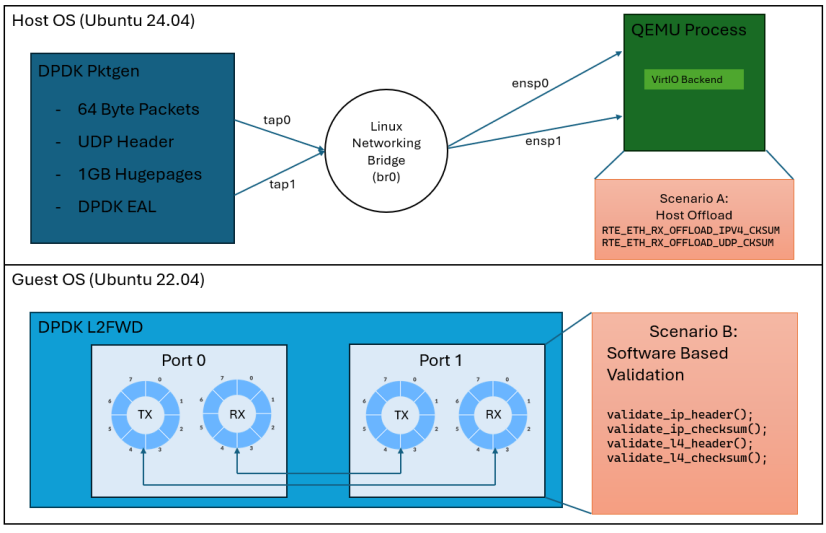


Figure 4: Guest vs. Host Validation Architecture.

The final research question will focus on applying malicious and/or erroneous packets to the first

research question. Instead of generating valid checksums and packet headers, this test will measure how much malformed packets affect the performance of DPDK. We are concerned and hypothesized that malformed kernel packets affect cache lines, IOTLB lookups, and other architecture-level penalties. Furthermore, this can be used along with the the first research question to further measure how a variety of packets (malformed or not) affect DPDK performance with different levels of packet verification.

```
generate_packets_malicious(
  count, source_port, malicious_type, malicious_percentage
){

  packets = alloc(count)
  for i = 1 to count {
     x = &packets[i];
     config_header(x);
     config_l3(x);
     config_udp(x);
     // -- Malform Packets --
     make_malformed = random_check(malicious_percentage);
     malform_type = make_malformed ? MALFORM_NONE : malicious_type;
     switch malform_type:
         case MALFORM_IP_CKSUM:
             ip_hdr.hdr_checksum = BITWISE_NOT(rte_ipv4_cksum(ip_hdr))
         case MALFORM_WRONG_IP_LEN:
             ip_hdr.total_length = 1500
             udp_hdr.dgram_len = 1500 - L3_SIZE
         case MALFORM_JUMBO_CLAIM:
             ip_hdr.total_length = 9000
             udp_hdr.dgram_len = 9000 - L3_SIZE
         case MALFORM_NONE:
             ip_hdr.hdr_checksum = rte_ipv4_cksum(ip_hdr)
             udp_hdr.dgram_cksum = rte_ipv4_udptcp_cksum(ip_hdr, udp_hdr)

  }
}
```

Figure 5: Generating Malicious Packets

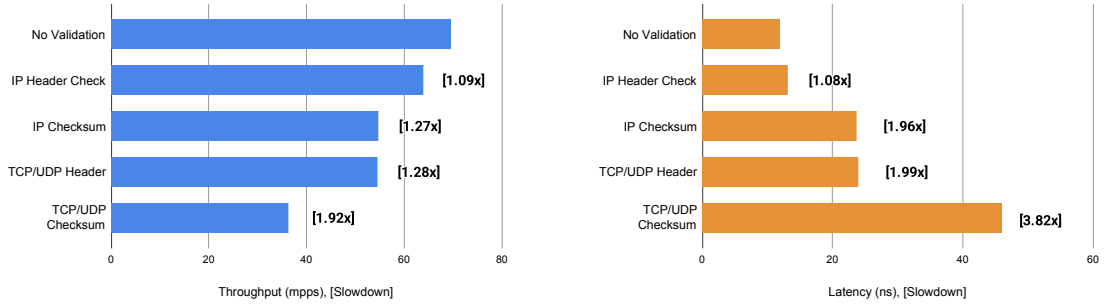
These experiments systematically quantify the cost of security when using DPDK. We measure the security-performance tradeoff from multiple lens and perspectives in order to gain a thorough understanding of how DPDK is affected under threat. First, we consider the cost of validating packets that is not present in native DPDK applications. Then, we measure the difference of offloading validation to the host machine as opposed to VM validation, which is extremely important for cloud-based DPDK applications. Finally, we remove validation and measure how malicious or malformed packets affect DPDK. This breadth and variety of experiments allows us to see multiple different ways DPDK could be degraded and exploited as well as provide developers granular level understanding of security cost for their applications.

4 Results

4.1 Fine-Grained Packet Validation

This section details the results shown by various levels of fine-grained packet validation and security done by default in the kernel. Shown by Figure 5 and Figure 6, we can see that trivially higher levels of validation result in lower throughput and higher latency. However, it is important to note how checksum validation is significant compared to simple layer 3 and layer 4 Header parsing. Finally, throughput and latency, seen as orthogonal forms of performance, are directly inverse in Figures 5 and 6. Overall, by using maximum levels of validation (i.e. layer 4 checksum) we can see 1.9x slowdown of throughput and a 4x slowdown of latency in DPDK performance.

The significant slowdown of checksum validation is mainly due to the difference in algorithmic complexity in TCP/UDP checksum validation. The IP checksum only requires a 20-byte IP header and is a fixed, fast operation. Conversely, the TCP/UDP checksum is calculated over the entire packet payload, and is a much more complex algorithm. Specifically, it requires the creation of a pseudo-header which allocates more memory and takes more time. This is shown through the 1.92x slowdown versus just the 1.27x slowdown for IP checksum.



(a) Throughput (b) Latency

Figure 6: Throughput and latency drop significantly with increased packet validation

It is important to note that latency measurements mean that the cores running DPDK applications were manually isolated, resulting in a constant maximum CPU utilization.

Considering a threat model where packets may be malicious or erroneous, we treat layer 3 and 4 checksum verifications as a default baseline for security as it is part of the standard protocol seen by most applications. However, applications that only pass information between a trusted network do not need the checksum verification, and can save upwards of 3.82x performance in latency by not utilizing these protocol-level security mechanisms. Ultimately, we leave this choice up to the end developer and their application-specific needs.

4.2 Guest vs. Host Validation Offloading

Exhaustive results have shown that validating packets on the guest or the host do not provide statistically significant differences in a majority of metrics. The initial hypothesis was wrong, but we provide a systematic reasoning for why this is and the insights provided by it.

After extensive analysis, we believe that the offloading on the host versus on the guest have equal but opposite costs that cancel each other out. The time taken to context switch to the Host and validate the packets is roughly equivalent to the time taken to perform a high-level software-based validation. Software validation is more expensive than hardware, but it is just as expensive as context switching and copying memory to have the validation on the hardware.

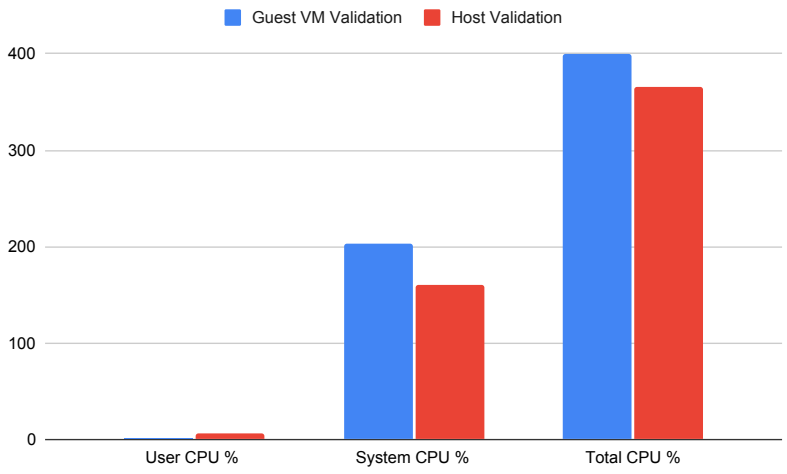


Figure 7: Offloading Packet Validation on Host Increases CPU Utilization.

This result is shown by the CPU utilization in Figure 7, the only metric that provided some significant result. Guest VM validation utilizes more of the CPU as it performs the validation, whereas

host validation is done on the NIC which results in less CPU utilization. Furthermore, a software-based validation done on the guest VM is another reason for the utilization difference.

Another reason CPU utilization is shown with a difference is that the VM is not isolated on a core. There must be context switching within the VM during this process. Utilization is minimized if the validation is offloaded to the host rather than a software based validation.

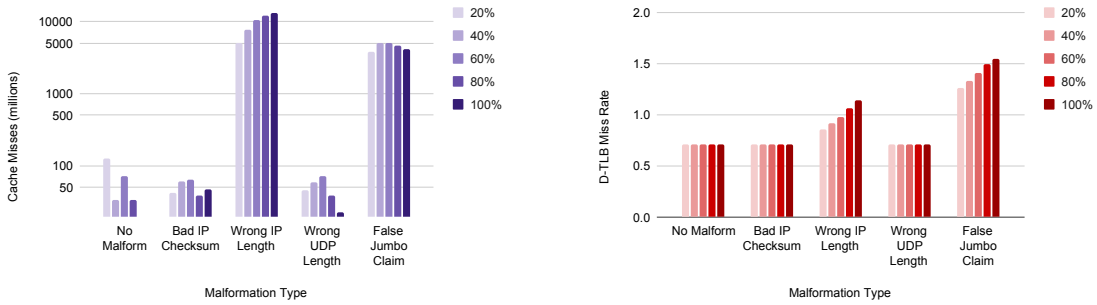
Other results such as latency and throughput show no statistically significant difference, and are thus not reported here.

From a security perspective, we consider two different threat models that result in different underlying designs for cloud-based DPDK applications. First, we only consider threat from incoming packets, not from the underlying VM. In this threat model, we claim no difference between validation done on the host versus the VM. Our second threat model considers a malicious VM, which could be seen in hijacked cloud environments. In that situation, offloading to host will be significantly more secure.

4.3 Malicious Packet Evaluation

Malicious packets have mixed results on performance of DPDK applications with no validation. Ultimately, the type of malicious packet is what makes the most difference in whether it will degrade DPDK performance or not. For instance, having a malformed or malicious IP checksum results in no significant difference in cache misses and data TLB misses.

On the other side, a false jumbo claim (claiming to have a large packet size despite not) results in 100x more cache misses than no malformation. This is most likely due to the CPU prefetching done by DPDK (through the `rte_prefetch0()` command) when it reads there is a large packet, but having misses due to the packet actually being small. Specifically, the CPU spends a long time fetching lines of what it expects to be valid packet data. This not only evicts valid and useful cache lines, but also pollutes the cache with useless data. This twofold effect is seen by the 100x difference in cache misses as well as the 2x difference in data TLB misses. The TLB misses indicate that the CPU assumes the packet size is spanning multiple pages of virtual memory, further proving how it has such an adverse affect on DPDK performance. This is again seen through packets that have the wrong IP length, which can again degrade prefetching or cache-line alignment. Overall, we see that misaligned or unexpected sizes of packets provides the most degradation of DPDK performance. Simply having bad data such as wrong checksums does not invalidate DPDK performance significantly.



(a) Cache Misses (total)

(b) Data TLB Misses (Rate)

Figure 8: Certain types of malicious packets have adverse affects on DPDK performance

The proposed threat model for this experiment is for an attacker to be able to send malicious packets into the network. They do not have access to the underlying machine and cannot execute code. Instead, the attacker's goal is to take advantage of the malicious packets into to significantly degrade performance of the DPDK application. These microarchitectural side-effects can turn into extreme slowdowns in the underlying application, which is what the attacker will exploit. To combat this, we again treat the validation used in section 4.1 as a standard baseline for packet validation. Systematic changes to the software such as blocking IPs or MACs are a way to retain high-performance from malicious packets.

Overall, These results present great insight into how attackers can create malicious packets for DPDK application. Rather than have malicious intent with the packets themselves, performing DoS attacks or flooding servers can have much greater effects when using malicious packets such as wrong IP length or false jumbo claims.

5 Related Work

An alternate approach to the same overarching problem enhances IPSec with DPDK [OLL20]. IPSec is a widely used standard that provides security by encrypting every incoming and outgoing packet at the kernel level. Due to IPSec being a well-established standard this can be used in many secure applications that would benefit from some performance gain. However, this work provides less freedom for the end user or application, as IPSec is an established software, not one someone can directly modify. Our work proposes a more granular and fine-grained security, and allows the user to directly modify the security-performance tradeoff of their application.

Another approach claims that software-based or even CPU based security, no matter how optimized, does not match hardware performance. Instead, they propose SmartNICs, specialized NICs that can perform IPSec encryption and encapsulation with near-zero overhead on the CPU [FHL25]. By utilizing hardware level security, they are allowed to perform security operations and have effectively the same network speed, due to the main bottleneck still being software. These works provide a beneficial solution for high-performance systems that can be used in industry applications, but do not consider smaller systems where additional costs are not feasible. Furthermore, it constrains the limited availability of NIC memory (SRAM).

Other works use a different methodology to bypass the kernel in order to extract performance while still maintain a higher level of security than that of DPDK. For instance, express data path (XDP) allows user-level programs to directly attach to the NIC, but have memory still managed by the kernel [HJBB+18]. Furthermore, if a packet needs extensive processing (such as L3/L4 checksum validation), it can simply be passed off to the kernel with minimal work from the user. This provides another approach than using custom validation through the extended Berkeley Packet Filter integrated in DPDK applications [SAD22]. However, this application does not provide the quantified granular level security shown by our work.

Using kernel bypass techniques in industry-applications is often hindered due to large amounts of code refactoring as well as an extensive check over an applications' security and performance demands. Works such as Junction [FCS+24] make kernel bypass techniques practical for cloud applications providing compatibility with native Linux binaries. This allows high-level applications (e.g Python, Java) to make system calls that will bypass the kernel. In this abstraction, they also provide significant memory management and scheduling optimizations. An important benefit of this approach is that it reduces the attack surface of the kernel bypass while also providing significant speedups. This work does not measure a security-performance tradeoff but instead presents their software as an end-user tool.

6 Conclusions

Our work shows that kernel bypass techniques result in an insignificant slowdown when implemented with the same security measures that the kernel networking stack uses by default. This is done through three structured experiments that cover the breadth of security and performance in DPDK applications. Furthermore, we quantify this gap with many fine-grained options of control for DPDK developers wanting to add security to their applications. Being able to quantitatively choose the security-performance tradeoff for a specific application provides a novel way to develop high-performance networking applications.

First, we show that increased packet validation (those ran by default in the Linux DPDK) provides a 1.92x slowdown in throughput and a 3.82x slowdown in latency, a significant cost for developers who are building high-performance network applications. We provide this to show the cost of security as well as allow developers to choose whether they need every level of packet validation or not depending on the application specific needs.

Second, we show that guest vs host validation has no statistically significant difference for the application, and we claim that from a security perspective guest versus host validation does not present any security threats from the packets, but could present some security threats if the underlying VM could be malicious (such a hijacked cloud environment).

Finally, we show that malformed and malicious packets provide a significant degradation to DPDK performance, which is an important consideration for developers building high-performance networking applications. Malformed packets that have the wrong size such as false jumbo claims or wrong IP lengths can cause upwards of 100x more cache misses and 2x more data-TLB misses, which can make DoS or server flooding attacks significantly worse than they already are.

Future works include more levels of security integrated into DPDK applications. For instance, packet encryption or utilizing security protocols such as TLS and HTTPs can provide more insights into how to best integrate DPDK applications while considering security. Another avenue is to further understand and analyze DPDK in various cloud environments such as serverless edge computing or confidential virtual machines.

Ultimately, this work shows that high-performance networking applications do not have to sacrifice security in order to remain competitive. Security can be implemented in a more customized and controlled manner in order to still have high throughput, low latency, or low CPU utilization. By modifying features such as levels of validation, guest vs. host validation offloading, or even worst-case packet handling, the future of security and performance is not orthogonal.

References

[ASVT18] A. Al-Sbo, M. Vorbrodt, and I. Toivanen. A High-Performance Implementation of an IoT System Using DPDK. *MDPI*, 8(4):550, 2018.

[BLM+18] David Barach, Leonardo Linguaglossa, Damjan Marion, Pierre Pfister, Salvatore Pontarelli, and Dario Rossi. High-Speed Software Data Plane via Vectorized Packet Processing. *IEEE Communications Magazine*, 56(8):97–103, 2018.

[FCS+24] Joshua Fried, Gohar Irfan Chaudhry, Enrique Saurez, Esha Choukse, Inigo Goiri, Sameh Elnikety, Rodrigo Fonseca, and Adam Belay. Making kernel bypass practical for the cloud with junction. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 55–73, Santa Clara, CA, April 2024. USENIX Association.

[FHL25] Mohammed Zain Farooqi, Masoud Hemmatpour, and Tore Heide Larsen. In-network memory access: Bridging smartnic and host memory, 2025.

[GEW+15] Sebastian Gallenmüller, Paul Emmerich, Florian Wohlfart, Daniel Raumer, and Georg Carle. Comparison of Frameworks for High-Performance Packet IO. *2015 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*, pages 29–38, 2015.

[GKCS22] Jose Gomez, Elie Kfoury, Jorge Crichigno, and Gautam Srivastava. A survey on tcp enhancements using p4-programmable devices. *Computer Networks*, 212, 05 2022.

[HJBB+18] Toke Høiland-Jørgensen, Jesper Dangaard Brouer, Daniel Borkmann, John Fastabend, Tom Herbert, David Ahern, and David Miller. The express data path: fast programmable packet processing in the operating system kernel. In *Proceedings of the 14th International Conference on Emerging Networking EXperiments and Technologies*, CoNEXT '18, page 54–66, New York, NY, USA, 2018. Association for Computing Machinery.

[HZLM16] Hai Huang, Dongyao Zhu, Kai Lu, and Chunhai Ma. DPDK-based Virtual Network Functions Acceleration in OpenStack. In *2016 12th International Conference on Mobile Ad-Hoc and Sensor Networks (MSN)*, pages 181–188, 2016.

[OLL20] Heekuk Oh, Younghee Lee, and Bong-Jae Lee. IPsec for DPDK-enabled Virtual Network Function with High Throughput and Security. *Future Generation Computer Systems*, 104:344–351, 2020.

[Riz12] Luigi Rizzo. netmap: a novel framework for fast packet I/O. *USENIX Annual Technical Conference (ATC '12)*, (186):1–12, 2012.

[SAD22] Husain Sharaf, Imtiaz Ahmad, and Tassos Dimitriou. Extended berkeley packet filter: An application perspective. *IEEE Access*, 10:126370–126393, 2022.

[SAPK23] Matheus Stolet, Liam Arzola, Simon Peter, and Antoine Kaufmann. Virtuoso: High Resource Utilization and µs-scale Performance Isolation in a Shared Virtual Machine TCP Network Stack. *arXiv preprint arXiv:2309.14016*, 2023.

[SLW+20] Yu Sun, Kexin Li, Haoyuan Wang, Junfang Su, and Jianshe Wang. NetDP: A DPDK based High-Performance Network Traffic Processor. *IEEE Access*, 8:109407–109418, 2020.

[SN16] Rinku Shah and Priyanka Naik. Kernel-bypass techniques for high-speed network packet processing CS-744. Presentation Slides, 2016.

[SRLBA18] Gustavo Sutter, Mario Ruiz, Sergio Lopez-Buedo, and Gustavo Alonso. Fpga-based tcp/ip checksum offloading engine for 100 gbps networks. In *2018 International Conference on ReConFigurable Computing and FPGAs (ReConFig)*, pages 1–6, 2018.

[VT23] M. Vorbrodt and I. Toivanen. *Analyzing the Performance of Linux Networking Approaches for Packet Processing : A Comparative Analysis of DPDK, io_uring and the standard Linux network stack*. Dissertation, Linköping University, 2023.

[Z+18] Jialong Zhou et al. XDP/DPDK-based High-speed Forwarding with Efficient Connection Management. In *2018 IEEE 26th International Conference on Network Protocols (ICNP)*, pages 1–10, 2018.